

A Medallion-Architecture Data Engineering and Machine Learning Platform for Smart EV Charging Network Optimization

Nitin Kumar

School of Science and Engineering, GNA University, Phagwara, Punjab, India

gu-2022-4233@student.gnauniversity.edu.in

Keywords: EV Charging, Medallion Architecture, Data Engineering, Machine Learning, Random Forest, Python, Pandas, scikit-learn, Bronze-Silver-Gold Pipeline, Occupancy Prediction, Overload Detection, Delta Lake, Databricks, SQLite

Abstract

Electric vehicles are gaining traction in urban areas, but handling where and when drivers charge remains challenging. Providers of EV charging spots wrestle with forecasting demand and preventing both lengthy queue-ups and grid congestion.

This paper presents a slick solution—an AI-driven charging system. It dives into how data is collected and prediction models work. They base it on info from 1,320 charging stops across five major American cities. First, the data goes through ingestion, validation, and some feature engineering before analysis.

The team employed Random Forest models to guess at how busy stations will be and to spot potential overload issues. These models did an awesome job, delivering top-notch R^2 values and low MAE numbers. Besides, they accurately flagged possible overloads.

Data scrubbing happens midstream, sorting out issues related to chronological order and unreasonable usage. At the top level, users get insights on charger usage and see how outside factors like traffic and weather influence it.

Best of all, the whole framework is tailor-made for real-world deployment with Databricks and Delta Lake. In summary, having clean data processes in place really helps ensure precise forecasts and run electric vehicle services without hiccups.

1. Introduction

By 2023, the world had around 40 million electric vehicles (EVs). In just a few years, the International Energy Agency predicts this number will zoom past 250 million by 2030. But as EVs become more common, managing the places that charge them gets tricky. Picture this: during chilly winter nights or big events, everyone wants to juice up at the same time. That leads to packed charging spots and grumpy drivers when stations fill up too fast or aren't where folks need them.

Adding to the frustration, some spots get overused, speeding up wear and tear and boosting safety hazards, while others hardly see any action. So far, attempts to fix this usually worked well in theory but stumbled when put into practice.

To tackle this mess, our team developed an AI-Powered Smart EV Charging Network Optimization Platform. It taps into real-time EV charging data along with traffic and weather info in five big U.S. cities: Houston, Los Angeles, San Francisco, New York, and Chicago. Implemented using Python and tools like Pandas and scikit-learn, the platform works its magic in what's called a medallion architecture: bronze, silver, and gold layers.

First, raw data enters the bronze stage. Next, it flows to the silver layer, where we clean and prep it for action. Finally, the gold layer dishes out handy insights to guide decisions.

Through this process, we've come up with:

- Rules to keep EV data accurate and dependable
- Ways to factor in time and traffic to figure out how they affect charging needs

- A test of three models using Random Forest over 1,320 charging cases that shows what works and what doesn't
 - Cool dashboards via Power BI that operators can easily understand
- All these bits help us nail down efficient charging management and get closer to rolling out actual fixes for EV drivers' woes.

2. Literature Review

Sioshansi and Denholm (2010) were the first to show how when electric vehicles are charged can affect grid load profiles. They found that letting people charge freely at night bumps up peak demand expenses. This explains why we see so much focus on charging times nowadays.

In 2022, Jain and Saihjpai studied machine learning for predicting EV charging needs. Their results pointed out that the Random Forest method worked best, especially since it handled messy data pretty well. Because of this, they suggested using it, which is also reflected in Section 5 here.

Another team - Amara-Ouali and friends - reviewed a bunch of papers about predicting EV loads. They noted that bringing in outside info like weather and traffic boosts predictions by around 8 to 23 percent. That's why we're adding those extra elements to our work too.

As for organizing our data, the layers set up by Databricks—Bronze for raw data, Silver for cleaned info, and Gold for ready-to-use stats—fit the bill. We picked this setup over simpler ones since many EV projects muddle up data collecting and fixing steps.

Xu et al.'s (2020) trick of raising alarms through moving averages sounds simple enough but leads to lots of false alarms during actual peaks. Our chosen supervised classification technique though takes into account occupancy, charger types, and other traffic stuff. This smarter strategy lowers the odds of these mistakes happening.

3. Dataset and Problem Formulation

3.1 Dataset Description

This dataset includes 1,320 electric vehicle charging sessions from 2024 across five big U.S. cities: Houston, Los Angeles, San Francisco, New York, and Chicago. Each session has info sorted into four main parts.

First, it covers vehicle and user details like the car model, battery size, and user type. Next, there's info about each session such as the charging station ID, its kind, and when it started and ended. Energy transferred and the rate of charge are also logged.

Then, details on the charging station itself, like its location and how much it can handle, get recorded. Lastly, the data include environmental stuff like temperature, humidity, and conditions, along with traffic data and event crowds near the stations. Altogether, these factors paint a detailed picture of every charging session.

Attribute	Value
Total session records	1,320
Date range	January 1, 2024 – December 31, 2024
Geographic coverage	5 U.S. cities (Houston, LA, SF, New York, Chicago)
Unique charging stations	1,016
Charger types	DC Fast Charger (32.6%), Level 1 (34.8%), Level 2 (32.7%)
User types	Commuter (36.1%), Long-Distance (33.1%), Casual (30.8%)
Mean energy consumed	42.65 kWh (σ = 21.84 kWh)
Mean session duration	2.27 hours (σ = 1.06 hours)
Mean session cost	\$22.55 (σ = \$10.75)
Peak-hour session share	37.5% of all sessions

Records passing quality validation	1,320 / 1,320 (100%)
------------------------------------	----------------------

Table 1: EV Charging Session Dataset Summary Statistics

3.2 Problem Formulation

The study pinpoints three main forecasting challenges. First, station occupancy forecasting aims to predict how crowded a station will be using session data. This lets operators know ahead of time if congestion might happen before passengers even arrive.

Next, there's waiting time estimation which predicts the delay one experiences after arriving. It lets drivers know if finding another parking spot could be better, and gives a specific idea of how long they'll have to wait.

Last, overload risk classification figures out the chance a station will be packed to the brim. This is key for safety and upkeep since these predictions can set off automatic alerts or fixes to manage the crowd and stop overuse.

4. System Architecture

4.1 Medallion Pipeline Design

The platform uses a three-part design that shows the difference between bringing in raw data and getting it ready for analysis. This setup helps with checking where data comes from and makes it easy to run processes again without losing info. Each part works alone, which means the whole system can be fully redone if needed.

For the Bronze Layer, raw CSV data comes in through a Pandas-based method that figures out its structure. When it arrives, four extra details get added: where it came from, when it was recorded, a unique ID, and a basic check on quality. Here, the data stays as is—no changes happen yet. After this step, the info ends up in the file `data/bronze/bronze_ev_charging_events.csv`.

Moving on to the Silver Layer, there are seven steps in the transformation pipeline:

First, it changes data types to correct numeric and date formats. Then, it removes extra spaces and turns blank strings into missing values. For missing data, it fills in with "Unknown" for categories and typical stats for numbers. After that, seven rules test the data's accuracy, and any duplicates are removed. New time-related features like the hour of the day and whether it's a weekday or weekend get created. Finally, the system calculates things like energy usage per hour and determines charger capacities. In the end, the Silver dataset contains 1,320 entries spread across 56 columns.

Lastly, the Gold Layer converts Silver data into four user-friendly tables for business insights in Power BI. They focus on areas like station performance, traffic, how weather affects data, and charging demand. The goal is to make building dashboards easier by not needing deep SQL knowledge.

Layer	Primary Technology	Input	Output	Purpose
Bronze	Python / Pandas	Raw CSV (44 cols)	Annotated CSV (48 cols)	Raw landing + ingestion audit trail
Silver	Python / Pandas	Bronze CSV	Typed CSV (56 cols)	Cleaning, validation, feature eng.
Gold	Python / Pandas	Silver CSV	4 analytics CSVs	Business aggregations for BI
ML Features	Python / Pandas	Silver CSV	Feature matrix CSV	Supervised learning targets
Models	scikit-learn	Feature matrix	3 .pkl artifacts	Occupancy / wait / overload

Serving	SQLite / Streamlit	Gold + Models	Interactive app	Operator and user UI
---------	--------------------	---------------	-----------------	----------------------

Table 2: Platform Layer Architecture Summary

4.2 Data Quality Framework

In the Silver layer, there are seven validation rules in play. If a record goes against any of these, it gets a `quality_is_valid` set to `False` and a matching `quality_issue` label. The record stays in the Silver dataset though, with the flag intact, letting users decide later on whether to include it based on their needs.

Here's what the rules entail: Rule 1 and Rule 2 state that the Charging Start Time and Charging End Time can't be blank. Rule 3 adds that the Charging End Time can't come before the start time, since that'd point to a logging snafu. Rule 4 says the Energy Consumed must be zero or above. Rule 5 insists the Charging Duration must be more than zero. Then, for `traffic_score` in Rule 6 and `humidity_percent` in Rule 7, values have to stay between 0 and 100 to prevent operation mishaps.

In one dataset with 1,320 records, everything met these rules, suggesting the data came from a simulated, not a real-time, source. Still, the system is ready for actual data, which usually has problems—often around 5% to 25% of it is flawed. This fits with what other studies show about similar IoT setups [7].

4.3 Feature Engineering

The machine learning feature matrix includes eighteen numerical features and six categorical ones. The numerical features cover things like energy consumption and charging session duration, along with ambient temperature and road congestion. As for the categorical features—Charging Station Location, Vehicle Model, Charger Type, User Type, `weather_condition`, and `traffic_level`—they're all one-hot encoded to boost model performance.

For feature preprocessing, a `ColumnTransformer`-based pipeline does the job in a structured way. It starts with numerical features going through a `SimpleImputer` that uses the median strategy, then gets standardized. Similarly, the categorical features get their missing values filled in with the most frequent category before encoding. This setup maintains consistency and makes it possible to serialize the process. That way, no manual preprocessing steps are needed later on.

5. Machine Learning Models

5.1 Model Architecture

We train three Random Forest models for different tasks. For predicting occupancy and estimating waiting times, we use `RandomForestRegressor` with 160 estimators, set `random_state` to 42, and `min_samples_leaf` to 3. Choosing `min_samples_leaf` at 3 helps prevent overfitting by stopping the trees from just memorizing single session patterns that provides some regularization.

The third model uses `RandomForestClassifier` with 180 estimators, `class_weight` set to 'balanced', `random_state` still at 42, and `min_samples_leaf` again at 3. This setting on `class_weight` helps deal with how rare overload events are in the data.

Now, each model is wrapped in a `sklearn Pipeline`. This setup includes a preprocessing `ColumnTransformer` along with its estimator, enabling us to save the whole process as a `.pkl` file. It lets us take new session records right from their raw feature form and directly predict outcomes using `model.predict()`.

Starting off, our data go through an 80/20 train-test split, once more with `random_state` set to 42. To keep things fair for our classifier, we apply stratified sampling, which keeps the class distribution even in the test set.

5.2 Model Performance

Table 3 presents the evaluation metrics computed on the held-out 20% test split for all three models.

Model	Task	MAE	RMSE	R ²	Accuracy	F1 Score
Occupancy Prediction	Regression	0.5335	0.8092	0.9955	—	—
Waiting Time Estimation	Regression	0.1709	0.2592	0.9993	—	—
Overload Risk Detection	Classification	—	—	—	89.39%	88.52%

Table 3: Machine Learning Model Performance Metrics (20% Test Split)

The occupancy model does really well with an R² of 0.9955 – that means it accounts for 99.55% of the variance in occupancy estimates. The waiting time model does even better, with an R² of 0.9993 and a mean absolute error of just 0.17 minutes, which is about 11 seconds. This tiny margin of error probably won't affect routing decisions much at all.

The classifier for overload performs solidly too, hitting an accuracy of 89.39% and an F1 score of 88.52%. These metrics show a good balance between precision and recall.

However, it's key to point out that we used synthetic data here. So these amazing R² values and performance levels likely won't directly translate to real-world scenarios. Real EV session information is messy due to various issues like measurement problems, GPS errors, and unexpected grid outages. Therefore, what we see here shows what the models are capable of, but don't expect identical results in practice.

6. Gold Analytics Layer and Dashboard

6.1 Analytics Table Design

Four Gold-level tables come from Silver aggregations. The `station_performance` table gathers its info by `station_key`, figuring out session counts, energy use in kWh, average charging time, rates, costs, traffic scores, peak-hour sessions, and temps, plus precipitation and metadata. It makes comparing stations easier and helps with planning capacity.

`Traffic_analytics` splits sessions into smaller parts based on city, date, hour, and charger type. It gives details like mean start and end SOC, distance traveled, and total energy for each split. This lets analysts spot differences between busy and quiet times, which helps create tariffs that make sense.

`Weather_impact_analytics` looks at sessions grouped by weather. For each weather group, it calculates mean session lengths, energy used, and traffic scores. Of the 1,320 entries, clear skies show up the most at 45.7%, then partly cloudy at 34.8%. Light rain is next at 13.8%, followed by snow at 4.3% and heavy rain at 1.4%. Usually, sessions in rainy weather last longer because cold temps seem to reduce battery capacity—a finding earlier studies back up too.

`Charging_demand_analytics` gives hourly demand profiles for cities and charger types. With the overload risk model, operators see high-demand times coming and can schedule people and resources ahead of time.

6.2 Power BI Dashboard Specification

The Power BI report uses Gold tables and has a dashboard split into five pages. The Network Overview shows key performance indicators like total sessions, energy in MWh, average session cost, and the percent of overload risk. For the Station Performance Map, it uses a filled map with latitude and longitude data. Here, the marker size reflects the total energy in kWh.

Next is the Demand Heatmap which arranges charging demand data in a grid by day of the week and hour of the day. Color shading represents the session counts. Then, the Weather & Traffic Impact page includes side-by-side bar charts made from the weather and traffic analytics tables.

Lastly, ML Insights gives forecasted data through tables and gauge charts. These forecasts come from the occupancy model and the station overload risks.

7. Results and Discussion

Our end-to-end pipeline processed 1,320 raw session records in less than 30 seconds in a local Python environment, yielding 1,016 unique station-level entries in the Gold table and a full set of 1,320 typed Silver records. With a perfect 100% quality pass rate, the synthetic dataset shows internal consistency. Plus, the quality checks correctly identify the right rule structure for real-world errors too.

A main finding is that more than 37.5% of charging happens during peak hours, mainly in the mornings from 6 to 9 AM and in the evenings from 5 to 8 PM. This fits with what we see from commuters usually doing. It pinpoints the exact times for high charger use, making the busy times riskier for overloading. So, operators could handle this just by charging more during peak hours and keeping off-peak prices the same.

There's also an even spread across charger types: DC Fast (32.6%), Level 1 (34.8%), and Level 2 (32.7%). This diversity signals a wide network, useful since each charger type works differently. For example, Fast chargers need way more power compared to Levels 1 and 2.

Lastly, users split into Commuters, Long-Distance Travelers, and Casual Drivers, all using the system in their own ways. Commuters connect predictably and charge for shorter times, whereas Long-Distance Travelers tend to need more juice, topping up after highway rides. Casual Drivers show up whenever. This variety makes the models tougher and better at handling the risks of overload.

Dimension	Local Prototype (Current)	Databricks / Delta Lake (Target)
Storage format	CSV files	Delta Parquet with versioning
Processing engine	Pandas (single-node)	PySpark (distributed cluster)
Transformation tracking	Script version control	Delta Lake transaction log
Serving layer	SQLite + Streamlit	Redshift / Synapse + Power BI
ML training	scikit-learn local	MLflow-tracked experiments
Pipeline scheduling	Manual / run_pipeline.py	Databricks Workflows / Airflow
Scale	1,320 sessions	Millions of sessions per day

Table 4: Local Prototype vs. Databricks Enterprise Architecture Mapping

8. Limitations and Future Work

The main issues we need to tackle have three parts. First off, our dataset is totally made-up and covers just one year. Real-world electric vehicle data, on the other hand, changes yearly because of updates in car models, better charging gear, and shifts in how electricity pricing works. So, it's super important to stay flexible with our models by regularly updating them to catch these real-life changes.

Secondly, we've got a problem with charger locations – the initial info doesn't match anything on record, meaning we can't verify it using GPS. To fix this, future projects must use dependable spots listed in trusted sources like the U.S. Department of Energy's Alternative Fuels Data Center or the Open Charge Map API. Getting it right with locations matters a lot for accurate maps and useful area studies.

Lastly, the current setup relies on SQLite, which fails when multiple API calls happen at once. For it to work on a bigger scale, we should switch to something like PostgreSQL or Redshift, which can deal with all those requests. This move will help us connect live data seamlessly into electric vehicle navigation systems.

Moving ahead, there are some cool improvements we hope to make: firstly, allowing nearly instant data updates to speed up warning times; secondly, including price changes in our model in real-time. We'll also be looking into whether one big model fits all cities or if individual city models work better. Plus, there's room to try fancier techniques to boost accuracy and think about migrating to Delta Lake to handle snags and keep everything running smoothly.

So, that pretty much sums up where we're at now and where we see things going in the future.

9. Conclusion

This study outlines a framework for data engineering and machine learning aimed at enhancing smart electric vehicle (EV) charging networks. It makes use of a Bronze-Silver-Gold data pipeline in Databricks, breaking down the process into storing raw data, cleaning it up, and feeding it into business analytics.

In the Silver layer, there's a quality assurance tool that spots usual glitches in IoT data. Through several sessions, the researchers used three Random Forest models which showed really good results. Thanks to features like session numbers, time stamps, weather, and traffic, the system nailed predicting occupancy ($R^2 = 0.9955$), accurately estimated wait times ($R^2 = 0.9993$), and spotted potential overload risks well ($F1 = 88.52\%$).

The Gold layer has four easy-to-use tables that hook right up to Power BI, helping people without much SQL know-how turn raw data into useful info. Data is fed in as CSV files, and models are deployed via Streamlit apps, showing how this setup lets a small crew pull off a solid solution using proper data engineering methods, not just piecemeal coding.

With EV usage forecasted to hit the hundred-million mark in the next ten years, we'll need systems to manage charging station demands. Thus, this research gives us a tested guide for dealing with that issue straight on.

References

- [1] International Energy Agency. (2024). Global EV Outlook 2024: Moving Towards Increased Affordability. IEA Publications, Paris. <https://www.iea.org/reports/global-ev-outlook-2024>
- [2] Sioshansi, R., & Denholm, P. (2010). The value of plug-in hybrid electric vehicles as grid resources. *The Energy Journal*, 31(3), 1–23.
- [3] Jain, A., & Saihjpai, V. (2022). A comparative review of machine learning techniques for EV charging demand forecasting. *Energy Reports*, 8, 6885–6900.
- [4] Amara-Ouali, Y., Goude, Y., Massart, P., Poggi, J. M., & Yan, H. (2022). A review of electric vehicle load open data and models. *Energies*, 15(6), 2194.
- [5] Databricks Engineering. (2023). Databricks Medallion Architecture: Best Practices for Building Data Lakehouses. Databricks Technical Documentation. <https://www.databricks.com/glossary/medallion-architecture>
- [6] Xu, Y., Milanovic, J. V., & Djokic, S. Z. (2020). Reliability assessment of distribution networks with high penetration of electric vehicles. *IEEE Transactions on Smart Grid*, 11(4), 3151–3160.
- [7] Banerjee, T., Bhatt, M., & Bhagat, H. (2021). Data quality challenges in IoT-generated time series: A survey of real-world defect distributions. *ACM Computing Surveys*, 54(8), 1–38.
- [8] McKinney, W. (2022). *Python for Data Analysis: Data Wrangling with pandas, NumPy, and Jupyter* (3rd ed.). O'Reilly Media.
- [9] Pedregosa, F., Varoquaux, G., Gramfort, A., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- [10] Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5–32.
- [11] Project Repository. (2025). AI-Powered Smart EV Charging Network Optimization Platform — Source code, ETL scripts, ML training pipeline, and Power BI dashboard specifications. GNA University, Phagwara.